4-bit Array Multiplier from RTL to GDSII on SkyWater SKY130 with Open-Source Tools

Sudhanva Joshi (2023102022) Aditya Peketi (2024122001)
Digital VLSI Design — RTL-to-GDSII Assignment
Repository: https://github.com/SudhuJ/arrayMultiplier

***Abstract*—We design a 4-bit $\times$ 4-bit unsigned array multiplier in Verilog with registered inputs and output, verify it exhaustively, and take it through synthesis (Yosys), static timing analysis (OpenSTA) and place and route (LibreLane) on the SKY130 high-density standard-cell library. All timing is evaluated at the typical corner ($25\,^{\circ}\mathrm{C}$, $1.80\,\mathrm{V}$). The RTL and the gate-level netlist both pass all 256 test vectors. Synthesis produces 61 cells with an area of $757.0\,\mu\mathrm{m}^2$, $52.9\,\%$ of it flip-flops. STA gives $T_{\min} = 2.79\,\mathrm{ns}$ ($F_{\max} = 358.4\,\mathrm{MHz}$). The final layout, at a $3.5\,\mathrm{ns}$ clock, has 0 DRC errors, 0 LVS errors and $0.077\,\mathrm{ns}$ of setup slack, giving a post-route $F_{\max}$ of $292.2\,\mathrm{MHz}$.**

I. Design

A. Array structure

For every bit b_i of B there is a partial product $pp_i[j] = a[j]\cdot b[i]$, shifted left by i. Row 0 consists only of AND gates: it gives $p[0] = pp_0[0]$ and passes $r_0 = \{0, pp_0[3{:}1]\}$ down. Rows 1–3 are 4-bit ripple-carry adders. Full adder j of row i adds $pp_i[j]$, $r_{i-1}[j]$ and the carry $c_i[j]$, with $c_i[0] = 0$. The lowest sum bit of each row is a finished product bit, $p[i] = s_i[0]$, and $r_i = \{c_i[4], s_i[3{:}1]\}$ goes to the next row. Row 3 also gives $p[7{:}4] = \{c_3[4], s_3[3{:}1]\}$. The array therefore uses $N^2 = 16$ AND gates and $N(N-1) = 12$ full adders (Fig. 1).

B. RTL

`full_adder` is written with two `assign` statements, $sum = a \oplus b \oplus c_{\mathrm{in}}$ and $c_{\mathrm{out}} = ab + a\,c_{\mathrm{in}} + b\,c_{\mathrm{in}}$. `mult_array` registers a and b into a_r and b_r on every rising edge, builds the three adder rows with `generate` loops (12 `full_adder` instances, no $\star$ operator) and registers the 8-bit result into p. All 16 flip-flops have an asynchronous, active-low reset. An input applied before edge 1 is captured at edge 1, and the product appears in p at edge 2. The latency is therefore two cycles, and the throughput is one result per cycle.

II. Part A: RTL Design and Verification

A. Testbench

`tb_mult` generates a $10\,\mathrm{ns}$ clock, holds `rst_n` low for the first two cycles and then applies all 256 combinations of a and b. Inputs change on the falling edge. After two rising edges the testbench waits another half cycle and compares p with $a \times b$ (the $\star$ operator is used only here). The comparison uses `!==`, so an unknown output also counts as an error. The testbench prints a line for every mismatch and one summary line, and writes `mult.vcd`. The RTL result is:

```
PASS: 256 vectors checked, 0 errors
```

B. Error-detection check

We temporarily changed the sum in `full_adder.v` to `a ^ b`, which ignores the carry input. The same testbench then reports 95 mismatches, the first being $a = 3$, $b = 3$, $p = 5$ (expected 9):

```
FAIL: 256 vectors checked, 95 errors
```

The change was then undone, and the testbench passes again.

C. Hand-worked example

We used Sudhanva Joshi's roll number (2023102022). Its last digit is 2, so $A = 2 + 5 = 7$, and because $A = 7$ the assignment gives $B = 9$.

$$
\begin{array}{rll}
 & 0\,1\,1\,1 & A = 7 \\
\times & 1\,0\,0\,1 & B = 9 \\
\hline
 & 0\,1\,1\,1 & pp_0\ (b_0 = 1) \\
 & 0\,0\,0\,0 & pp_1\ (b_1 = 0),\ \text{shift } 1 \\
 & 0\,0\,0\,0 & pp_2\ (b_2 = 0),\ \text{shift } 2 \\
 & 0\,1\,1\,1 & pp_3\ (b_3 = 1),\ \text{shift } 3 \\
\hline
 & 0\,0\,1\,1\,1\,1\,1\,1 & P = 63
\end{array}
$$

In the array, $r_0 = 0011$.

- Row 1 adds $pp_1 = 0000$: $s_1 = 0011$, so $p[1] = 1$ and $r_1 = 0001$.
- Row 2 adds $pp_2 = 0000$: $s_2 = 0001$, so $p[2] = 1$ and $r_2 = 0000$.
- Row 3 adds $pp_3 = 0111$: $s_3 = 0111$ and $c_3[4] = 0$, so $p[3] = 1$ and $p[7{:}4] = 0011$.

Hence $P = 0011\,1111_2 = 63$.

D. Waveform

Fig. 2 shows the example in the RTL simulation. With $a = 7$ throughout, b changes from 8 to 9 on the falling edge at $2\,440\,\mathrm{ns}$. Rising edge 1 at $2\,445\,\mathrm{ns}$ (marker A) captures a and b into the input registers. Rising edge 2 at $2\,455\,\mathrm{ns}$ (marker B) captures the product, and p changes from 56 to 63. The latency is two clock cycles, or $20\,\mathrm{ns}$.

III. Part B: Logic Synthesis

The Yosys script `synth/synth.tcl` performs these steps:

1) read both RTL files;
2) run generic synthesis with the full adders flattened into one module;
3) map the flip-flops, and then the logic (with ABC), to the typical-corner Liberty library;

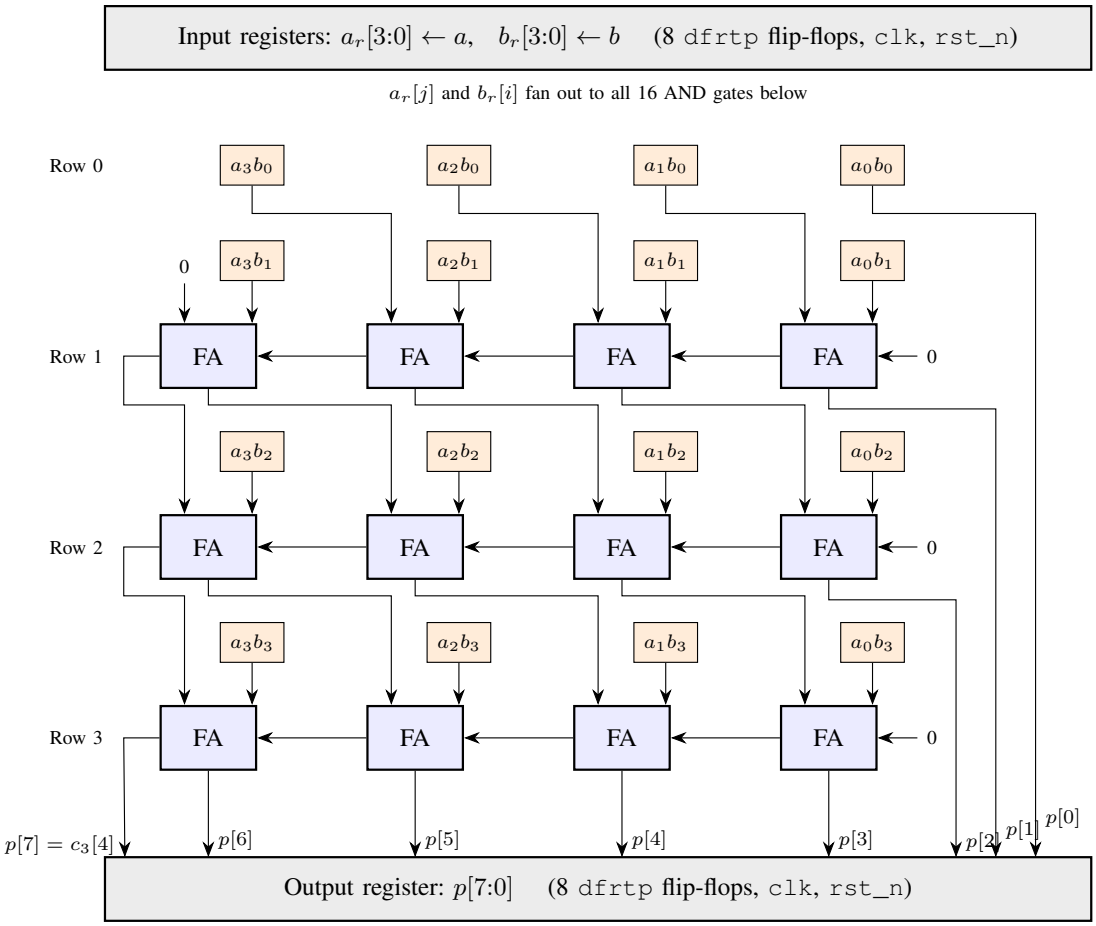


Fig. 1. Block diagram of `mult_array`: 16 AND gates ($a_jb_i = pp_i[j]$), 12 full adders in three ripple-carry rows, and the input and output registers (16 flip-flops). Each full adder takes $pp_i[j]$ on its right input and $r_{i-1}[j]$ on its left input; carries ripple from right to left.

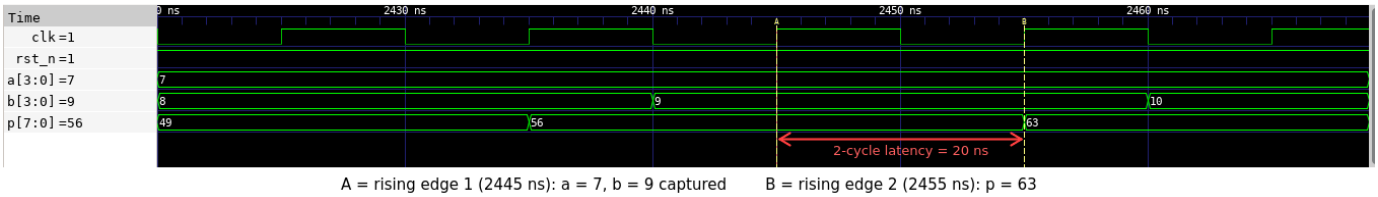


Fig. 2. GTKWave view of `clk`, `rst_n`, a, b and p (decimal) for the example 7×9. Markers A and B are the two rising edges of the 2-cycle latency.

4) remove unused wires;
5) write the cell and area report and the gate-level netlist.

Table I summarises the results. All cell names omit the library prefix `sky130_fd_sc_hd__`. The flip-flop cell `dfrtp_1` has an area of $25.024\,\mu\text{m}^2$.

We simulated the gate-level netlist against the SKY130 functional cell models with unit gate delays, using the same testbench. It passes all 256 vectors, so synthesis preserved the behaviour. The netlist also contains two `lpflow_isobufsrc_1` cells. ABC is free to pick them because the script does not exclude any library cells.

IV. Part C: Static Timing Analysis

The OpenSTA script `sta/sta.tcl` reads the typical-corner library and the netlist. It then defines the clock, sets 1.0 ns input and output delays, and reports the worst setup slack and the critical path. At $T = 10\,\text{ns}$ the worst slack is $7.213\,\text{ns}$, so

$$T_{\min} = 10 - 7.213 = 2.787\,\text{ns} \approx 2.79\,\text{ns},$$
$$F_{\max} = 1000/2.79 = 358.4\,\text{MHz}.$$

Rerunning at $T = 2.79\,\text{ns}$ gives $0.003\,\text{ns}$ of slack (Table II).

The critical path (Table III) runs from the register holding $a_r[1]$ to the register that captures $p[6]$. Five of its eight logic cells are recognisable full-adder cells: three `maj3` cells (carry)

TABLE I
PART B RESULTS

Metric	Value
Total number of cells	61
Number of flip-flops	16
Total cell area	$756.976\,\mu\text{m}^2$
Flip-flop area (16×25.024)	$400.384\,\mu\text{m}^2$
Flip-flop area (share of total)	$52.89\,\%$
Three most-used cell types	
`dfrtp_1`	16
`nand2_1`	7
`maj3_1` and `and2_0` (tied)	4 each
Netlist simulation	
`PASS: 256 vectors checked, 0 errors`	

TABLE II
PART C RESULTS (TYPICAL CORNER)

Metric	Value
Worst slack at $T = 10\,\text{ns}$	$7.213\,\text{ns}$
$T_{\min}$	$2.79\,\text{ns}$
$F_{\max}$	$358.4\,\text{MHz}$
Slack at $T = T_{\min}$	$0.003\,\text{ns}$
Critical path start point	`_91_` ($a_r[1]$)
Critical path end point	`_88_` ($p[6]$)
Cells on the critical path	8 logic, 2 flip-flops
Latency ($2 \times T_{\min}$)	$5.58\,\text{ns}$
Throughput	358.4 million results/s

TABLE III
CRITICAL PATH AT $T = 2.79\,\text{ns}$

#	Cell	Role	Delay (ns)
–	`dfrtp_1` (`_91_`)	launch, clock to Q	0.398
1	`clkinv_1`	partial products	0.073
2	`o311ai_0`	partial products, row 1	0.356
3	`a21oi_1`	row 1	0.153
4	`maj3_1`	FA carry	0.402
5	`xnor3_1`	FA sum	0.370
6	`maj3_1`	FA carry	0.368
7	`maj3_1`	FA carry	0.354
8	`xor3_1`	FA sum	0.178
–	`dfrtp_1` (`_88_`)	capture, setup time	0.135
	Data arrival time		2.652

and the `xnor3` and `xor3` cells (sum). The first three cells implement the partial products and the simplified first adder row, which ABC merged into complex gates.

V. PART D: PHYSICAL DESIGN

A. Configuration

Table IV lists the settings in `pnr/config.json`. The clock period is $1.2 \times T_{\min} = 1.2 \times 2.79 = 3.348\,\text{ns}$, rounded up to the next $0.5\,\text{ns}$. As in Part C, timing is signed off at the typical corner only, which the `STA_CORNERS` setting selects.

TABLE IV
LIBRELANE CONFIGURATION

Key	Value
`DESIGN_NAME`	`mult_array`
`VERILOG_FILES`	`full_adder.v`, `mult_array.v`
`CLOCK_PORT`	`clk`
`CLOCK_PERIOD`	$3.5\,\text{ns}$
`FP_SIZING`	`absolute`
`DIE_AREA`	$0, 0, 150, 150\ (\mu\text{m})$
`STA_CORNERS`	`nom_tt_025C_1v80`

TABLE V
PART D RESULTS (TYPICAL CORNER)

Metric	Value
`CLOCK_PERIOD` used	$3.5\,\text{ns}$
Number of runs needed	1
Core area	$17\,759.5\,\mu\text{m}^2$
Die area	$22\,500\,\mu\text{m}^2$
Cells after synthesis (Part B)	61
Cells after place and route	1882
Kinds of cells added	decap, fill, tap, buffers
Wirelength	$2\,495\,\mu\text{m}$
Setup slack	$0.077\,\text{ns}$
Post-route $F_{\max}$	$292.2\,\text{MHz}$
$F_{\max}$ from Part C	$358.4\,\text{MHz}$
DRC errors (Magic, KLayout)	0, 0
LVS errors	0

B. Results

The flow finished in a single run at $3.5\,\text{ns}$ with positive setup slack, 0 DRC errors and 0 LVS errors. Table V lists the final metrics from `pnr/metrics.json`. The post-route maximum frequency is

$$F_{\max} = \frac{1000}{3.5 - 0.077} = \frac{1000}{3.423} = 292.2\,\text{MHz},$$

$18\,\%$ lower than the $358.4\,\text{MHz}$ from Part C. The post-route critical path runs from the $a_r[0]$ register to the $p[6]$ register through eight logic cells. Four of them are full-adder cells: `xor3`, two `maj3` and `xnor3`.

C. Cells added by the flow

Table VI breaks down the 1882 instances. LibreLane re-synthesises the RTL itself and produces 62 logic and flip-flop cells. That is one more than Part B, because LibreLane's synthesis does not use the `lpflow` cells. Every other cell was added during physical design.

D. Layout

Fig. 3 shows the final GDS in KLayout.

- **Whole die:** the $150\,\mu\text{m}$ die, the standard-cell rows and the I/O pins on the die edges. The signal routing is concentrated in the middle of the core, where the 84 logic, flip-flop and buffer cells sit; the rest of the rows hold decap and fill cells.

TABLE VI
CELLS AFTER PLACE AND ROUTE

Cell kind	Count
Design logic and flip-flops	62
Decap (`decap_3` to `decap_12`)	1300
Fill (`fill_1`, `fill_2`)	253
Well taps (`tapvpwrvgnd_1`)	245
Timing-repair buffers (`buf_2`, `clkdlybuf4s50_1`)	19
Clock-tree buffers (`clkbuf_16`)	3
Total	1882

- **Zoomed view:** the rows bounded by horizontal power rails, the poly gates inside the cells, horizontal metal 1 and vertical metal 2 wires, and the vias between them.

E. A note on process corners

By default LibreLane signs off at nine process corners. Before we restricted signoff to the typical corner, exploratory runs with all corners met timing at the typical corner. They failed only at the slow corner (100 °C, 1.60 V), where the path is roughly twice as slow. The README records these runs. The assignment evaluates timing at the typical corner, so the reported run uses that corner only.

VI. ANSWERS TO THE QUESTIONS

Q1. Synthesis produced $61 - 16 = 45$ logic cells for the array, instead of $16 + 12 = 28$ blocks. The count is not one cell per block, because flattening dissolves the full-adder boundaries and ABC re-optimises the whole logic cone. Some effects add cells:

- Some full adders are split into a sum cell (`xor3`, `xnor3` or `xor2`) and a carry cell (`maj3`), or into several smaller gates.
- The library's full-adder cell `fa_1` has two outputs, which ABC cannot map to; it reports it as a multi-output cell and skips it. Each full adder therefore costs at least two cells.

Others remove cells:

- Adders with a constant-0 input ($c_i[0] = 0$, $r_0[3] = 0$) reduce to half adders.
- AND gates are merged into complex cells such as `a21oi`, `a22oi` and `o311ai`.

Q2. At $T_{\min} = 2.79\,\text{ns}$, the critical path runs from the input-register flip-flop holding $a_r[1]$ to the output-register flip-flop that captures $p[6]$. It has eight logic cells between the two flip-flops (Table III), five of them full-adder cells. The estimate of $2N = 8$ full-adder delays describes the unoptimised array. Here the first stages are simplified into three fast complex gates (half adders and merged AND logic), so only five full-adder stages remain. Those five cells take $1.672\,\text{ns}$, about $0.33\,\text{ns}$ each, and the three first-stage gates take $0.582\,\text{ns}$. The total logic delay of $2.254\,\text{ns}$ therefore corresponds to about seven full-adder delays, slightly below the estimate of eight.

Q3. Flip-flops take $52.89\,\%$ of the total cell area: 400.384 of $756.976\,\mu\text{m}^2$. A `dfrtp_1` with asynchronous reset is $25.0\,\mu\text{m}^2$, about seven times a `nand2_1` ($3.75\,\mu\text{m}^2$), because it contains two latches, a clock inverter and reset logic. The design needs 16 of them (8 input bits and 8 output bits), while the combinational array is only 45 small gates. The number of registers grows linearly with N, but the array grows with N^2, so the flip-flop share is largest for small multipliers.

Q4. The post-route $F_{\max}$ is $292.2\,\text{MHz}$, against $358.4\,\text{MHz}$ in Part C. Three effects lower it:

- The routed wires add resistance and capacitance, which are extracted after routing.
- The flow adds a $0.25\,\text{ns}$ clock uncertainty and a $5\,\%$ timing derate.
- The inputs are driven by a real cell and the outputs drive a real load.

Together these stretch the launch-to-capture data path from $2.652\,\text{ns}$ to $3.03\,\text{ns}$. The flow added five kinds of cells (Table VI), each for a reason:

- **Clock-tree buffers** (`clkbuf_16`) deliver the clock to all 16 flip-flops with low skew. They add $0.32\,\text{ns}$ of insertion delay, which is common to the launch and capture flip-flops.
- **Timing-repair buffers** fix slew and capacitance limits and improve timing.
- **Tap cells** tie the n-wells and the substrate to VPWR and VGND, preventing latch-up.
- **Decap cells** add capacitance between the supply rails, reducing supply noise when cells switch.
- **Fill cells** close the remaining gaps in the rows, so that the wells, implants and power rails are continuous, as the design rules require.

Q5. A DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths, spacings, enclosures, overlaps and densities on every layer. An LVS (layout versus schematic) check extracts the transistors and their connections from the layout and compares them with the netlist. It confirms that the layout implements exactly the intended circuit, with no shorts, opens or missing devices. Both must pass before fabrication:

- A DRC violation can make the chip impossible to manufacture reliably, for example through shorts, broken wires or low yield.
- An LVS mismatch gives a chip that can be manufactured but does the wrong thing.

Our final layout has 0 DRC errors (Magic and KLayout) and 0 LVS errors (Netgen).

Q6. For an 8-bit multiplier ($N = 8$):

- **Size:** the array needs $N^2 = 64$ AND gates and $N(N-1) = 56$ full adders, against 16 and 12 for $N = 4$. That is about 4.7 times as many full adders, so the logic area grows roughly with N^2.
- **Path length:** the critical path is about $2N = 16$ full-adder delays instead of 8, twice as long. The delay grows linearly with N.

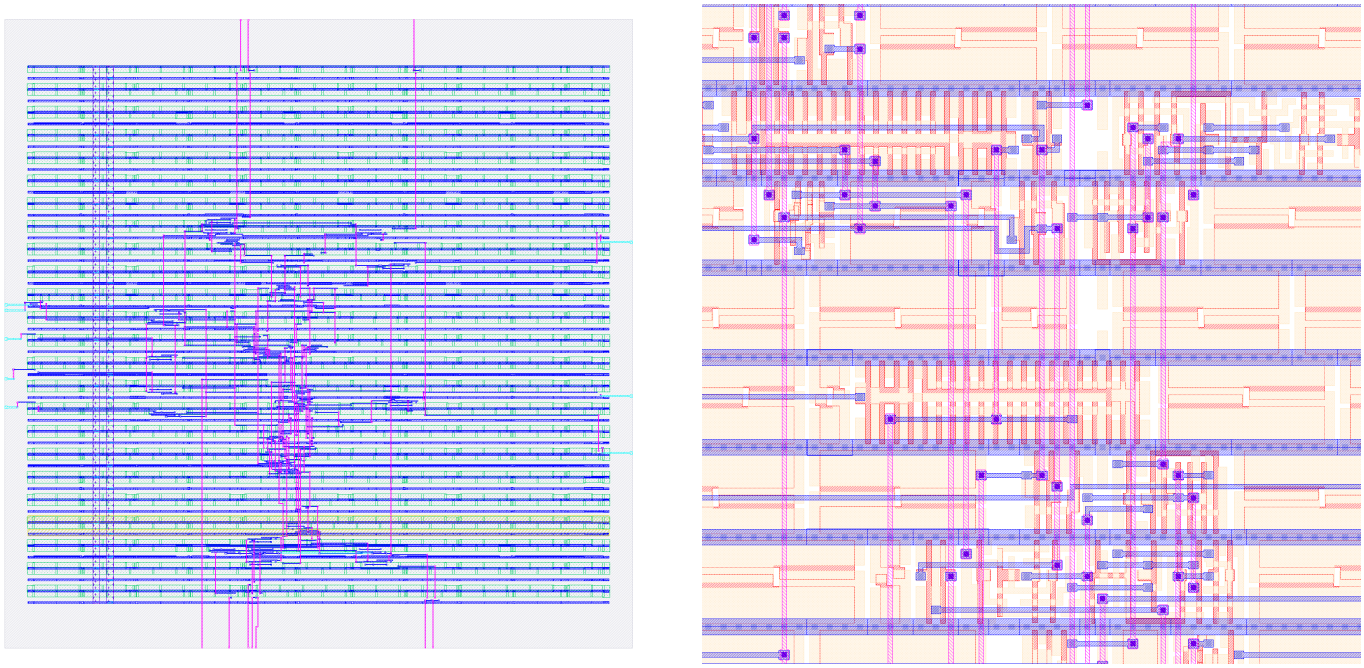


Fig. 3. Final layout in KLayout. Left: whole die ($150\,\mu\text{m} \times 150\,\mu\text{m}$; metal layers, n-well and boundary). Right: zoomed view of about $20\,\mu\text{m} \times 20\,\mu\text{m}$ (poly, local interconnect, metal 1, metal 2 and vias).

- **Delay estimate:** in Part C, the 4-bit logic delay is $2.652 - 0.398 = 2.254\,\text{ns}$. Scaling it by $16/8$ gives about $4.5\,\text{ns}$ of logic. Adding the clock-to-Q time ($0.40\,\text{ns}$) and the setup time ($0.14\,\text{ns}$) gives $T_{\min} \approx 5.0\,\text{ns}$, or $F_{\max} \approx 200\,\text{MHz}$, against $358\,\text{MHz}$ for the 4-bit design.
- **Registers:** the design would need $8 + 8 + 16 = 32$ flip-flops.

VII. Conclusion

We took the 4-bit array multiplier from RTL to a DRC- and LVS-clean SKY130 layout. It is functionally correct in both RTL and gate-level simulation. At the typical corner it runs at $358\,\text{MHz}$ before layout and $292\,\text{MHz}$ after routing, on a $150\,\mu\text{m} \times 150\,\mu\text{m}$ die. All source files, scripts and generated reports are in the repository at https://github.com/SudhuJ/arrayMultiplier, whose README lists every command, in order.